PDS Practical-6

Name: Vaibhav Bhusari

Div:SY-A

Roll No:A-11

Aim: Implementing Web Scrapping in Python with BeautifulSoup

```
pip install requests beautifulsoup4
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/di
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyt
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/pyt
```

```
import requests
from bs4 import BeautifulSoup
```

```
# Define the URL of the website you want to scrape
url = 'http://quotes.toscrape.com/' # A website specifically designed for sc

# Send an HTTP GET request to the URL
response = requests.get(url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    print(f"Successfully fetched content from {url}")
    # Get the HTML content of the page
    html_content = response.text
else:
    print(f"Failed to retrieve content. Status code: {response.status_code}"
    html_content = None
```

```
Successfully fetched content from http://quotes.toscrape.com/
```

```
if html_content:
    # Create a BeautifulSoup object
    soup = BeautifulSoup(html_          html.parser')
    print("HTML content parsed          ully with BeautifulSoup.")
    # You can print a small pa          prettified HTML to see the structure
    # print(soup.prettify()[:1000]) # Print first 1000 characters of prettif
else:
```

```
    soup = None
    print("Cannot parse HTML as content was not fetched.")
```

```
HTML content parsed successfully with BeautifulSoup.
```

```
if soup:
    # Extract the page title
    title = soup.title.text if soup.title else 'No Title Found'
    print(f"\nPage Title: {title}\n")

    print("Extracting quotes and authors:")
    # Find all div elements with class 'quote'
    quotes = soup.find_all('div', class_='quote')

    for quote in quotes:
        text = quote.find('span', class_='text').text
        author = quote.find('small', class_='author').text
        print(f"Quote: {text}\nAuthor: {author}\n")
else:
    print("Cannot extract data as BeautifulSoup object is not available.")
```

```
Page Title: Quotes to Scrape

Extracting quotes and authors:
Quote: “The world as we have created it is a process of our thinking. It cann
Author: Albert Einstein

Quote: “It is our choices, Harry, that show what we truly are, far more than
Author: J.K. Rowling

Quote: “There are only two ways to live your life. One is as though nothing i
Author: Albert Einstein

Quote: “The person, be it gentleman or lady, who has not pleasure in a good n
Author: Jane Austen

Quote: “Imperfection is beauty, madness is genius and it's better to be absol
Author: Marilyn Monroe

Quote: “Try not to become a man of success. Rather become a man of value.”
Author: Albert Einstein

Quote: “It is better to be hated for what you are than to be loved for what y
Author: André Gide

Quote: “I have not failed. I've just found 10,000 ways that won't work.”
Author: Thomas A. Edison

Quote: “A woman is like a tea bag; you never know how strong it is until it's
Author: Eleanor Roosevelt

Quote: “A day without sunshine                you know, night.”
Author: Steve Martin
```

Q1: Scrape tabular data from a webpage

Q2: Store scraped data in a CSV file

Q3: Handle missing or broken tags

Q4: Use requests with headers

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# Define the URL for a page with tabular data (e.g., Wikipedia)
tabular_url = 'https://en.wikipedia.org/wiki/List_of_countries_by_population_

# Add headers to mimic a browser request (Task 10)
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
}

# Send an HTTP GET request to the URL with headers
tabular_response = requests.get(tabular_url, headers=headers)

# Check if the request was successful
if tabular_response.status_code == 200:
    print(f"Successfully fetched tabular content from {tabular_url}")
    tabular_html_content = tabular_response.text
    tabular_soup = BeautifulSoup(tabular_html_content, 'html.parser')
    print("Tabular HTML content parsed successfully with BeautifulSoup.")

    # Find the main table(s) on the page. Wikipedia tables often have the cla
    # We'll try to find the first table with this class.
    table = tabular_soup.find('table', class_='wikitable')

    if table:
        # Extract table headers
        # Using .text.strip() to clean up header names
        headers = [th.text.strip() for th in table.find('tr').find_all('th')]
        print(f"Found table headers: {headers}")

        # Extract table rows (Task 6)
        table_data = []
        for row in table.find_all('tr')[1:]: # Skip the header row
            # Extract all 'td' and 'th' elements in the row for robustness
            row_data = [td.text.strip() for td in row.find_all(['td', 'th'])]
            table_data.append(row_data)

        # Handle missing or broken tags by ensuring consistent row lengths (T
        cleaned_table_data = [
        for row in table_data:
            if len(row) == len                    :
                cleaned_table_data.append(row)
            elif len(row) > len(headers):
```

```
                # If a row has more columns than headers, truncate it
                cleaned_table_data.append(row[:len(headers)])
            else:
                # If a row has fewer columns, pad with empty strings
                cleaned_table_data.append(row + [''] * (len(headers) - len(ro

        # Create a Pandas DataFrame
        df_countries = pd.DataFrame(cleaned_table_data, columns=headers)
        print("\nDataFrame created from tabular data:")
        display(df_countries.head())

        # Save the tabular data to a new CSV file (Task 7)
        output_tabular_csv_file = 'countries_by_population.csv'
        df_countries.to_csv(output_tabular_csv_file, index=False, encoding='u
        print(f"\nScraped tabular data successfully saved to '{output_tabular

    else:
        print("No table with class 'wikitable' found on the page.")

else:
    print(f"Failed to retrieve tabular content. Status code: {tabular_respons
```

```
Successfully fetched tabular content from https://en.wikipedia.org/wiki/List_
Tabular HTML content parsed successfully with BeautifulSoup.
Found table headers: ['Country or territory', 'Population(1 July 2022)', 'Pop

DataFrame created from tabular data:
```

	Country or territory	Population(1 July 2022)	Population(1 July 2023)	Change(%)	UN continentalregion[1]	st
0	World	8,021,407,192	8,091,734,930	+0.88%	–	
1	India	1,425,423,212	1,438,069,596	+0.89%	Asia	
2	China[a]	1,425,179,569	1,422,584,933	−0.18%	Asia	
3	United States	341,534,046	343,477,335	+0.57%	Americas	
4	Indonesia	278,830,529	281,190,067	+0.85%	Asia	

```
Scraped tabular data successfully saved to 'countries_by_population.csv'
```

Q4: Scrape data using class and ID aflributes

We've already seen examples of scraping by class (e.g., `div` with `class='quote'` or `table` with `class='wikitable'`). Let's find an element by its `id` aflribute on the original `quotes.toscrape.com` ere isn't a prominent `id` on that page, I'll find one on another commo Wikipedia for demonstration purposes.

```
# Using the already fetched html_content from http://quotes.toscrape.com/
if soup:
    print("Scraping by class attribute (already done for quotes):")
    # Example: Finding all div elements with class 'quote'
    quotes_by_class = soup.find_all('div', class_='quote')
    print(f"Found {len(quotes_by_class)} quotes using class 'quote'.")
    if quotes_by_class:
        print(f"First quote text: {quotes_by_class[0].find('span', class_='te

    print("\nNow attempting to scrape by ID attribute:")
    # Let's check for an element with a specific ID on the quotes page.
    # Often, IDs are used for unique page elements like headers, footers, or
    # The 'quotes.toscrape.com' page doesn't have many prominent IDs, so I'll
    # For instance, a common element found by ID might be a 'footer' or 'head
    # Let's look for a hypothetical element with id='footer' or id='header' o

    # Inspecting quotes.toscrape.com, there isn't a clear ID to target.
    # Let's try finding the navigation link to 'top ten tags' which has a cla
    # Instead, I'll try to find an 'a' tag (link) that has an 'href' attribut

    # For a more direct 'id' example, let's use the Wikipedia page, which oft
    print("\nUsing the Wikipedia page for a clear 'id' example:")
    if 'tabular_soup' in locals(): # Check if tabular_soup was successfully c
        # Wikipedia often uses IDs for sections. For example, 'References' or
        # Let's try to find the element with id='References'
        references_section = tabular_soup.find(id='References')
        if references_section:
            print(f"Found element with id='References': {references_section.t
        else:
            print("Could not find element with id='References' on the Wikiped

        # Let's try to find another common ID, for example, the main content
        main_content = tabular_soup.find(id='content')
        if main_content:
            print(f"Found element with id='content'. First H1 title: {main_co
        else:
            print("Could not find element with id='content' on the Wikipedia
    else:
        print("Tabular soup object not available for ID scraping demonstratio
else:
    print("BeautifulSoup object not available for scraping by class/ID.")
```

```
Scraping by class attribute (already done for quotes):
Found 10 quotes using class 'quote'.
First quote text: “The world as we have created it is a process of our thinki

Now attempting to scrape by ID attribute:

Using the Wikipedia page for a clear 'id' example:
Found element with id='References': References
Found element with id='content'. First H1 title: List of countries and depend
```


Conclusion

This practical session successfully demonstrated key concepts and techniques in web scraping using Python with requests and BeautifulSoup libraries. We covered the entire process, starting from fetching HTML content from a website, parsing it, and then extracting specific data.